Environmental Modelling and Software 169 (2023) 105809

Contents lists available at ScienceDirect

Environmental Modelling and Software

journal homepage: www.elsevier.com/locate/envsoft

System for automated Quality Control (SaQC) to enable traceable and reproducible data streams in environmental science

Lennart Schmidt [a,b,*], David Schäfer [a,b,1], Juliane Geller [a,b], Peter Lünenschloss [a,b], Bert Palm [a,b], Karsten Rinke [c], Corinna Rebmann [d], Michael Rode [e,f], Jan Bumberger [a,b,g]

[a] Research Data Management - RDM, Helmholtz Centre for Environmental Research - UFZ, Permoserstraße 15, Leipzig, 04318, Germany
[b] Department of Monitoring and Exploration Technologies, Helmholtz Centre for Environmental Research - UFZ, Permoserstraße 15, Leipzig, 04318, Germany
[c] Department of Lake Research, Helmholtz Centre for Environmental Research - UFZ, Brückstraße 3a, Magdeburg, 39114, Germany
[d] Department of Computational Hydrosystems, Helmholtz Centre for Environmental Research - UFZ, Permoserstraße 15, Leipzig, 04318, Germany
[e] Department of Aquatic Ecosystem Analysis, Helmholtz Centre for Environmental Research - UFZ, Brückstraße 3a, Magdeburg, 39114, Germany
[f] Institute of Environmental Science and Geography, University of Potsdam, Am Neuen Palais 10, Potsdam, 14469, Germany
[g] German Centre for Integrative Biodiversity Research (iDiv) Halle-Jena- Leipzig, Puschstraße 4, Leipzig, 04103, Germany

ARTICLE INFO

Dataset link: https://doi.org/10.5281/zenodo.8092119

Keywords:
Data management
Quality control
Quality assurance
Anomaly detection
Sensor data
FAIR

ABSTRACT

Environmental sensor networks produce continuously increasing volumes of raw data that need to be transformed into usable data for monitoring ongoing environmental changes and decision-support. The crucial challenge is providing data in real-time, which requires the rigorous automation of quality control (QC) workflows using suitable software tools. We present the System for automated Quality Control (SaQC), a software package for the automated quality control of environmental time series data that is universal and that can be expanded in its set of domain-agnostic QC and processing functionalities, while at the same time being user-friendly in its low-code configuration environment. Two applications present the configuration of basic and advanced quality control applications using SaQC. We also elaborate on the explicit user controls over the handling of quality flags and how SaQC can be used to make QC-workflows traceable and reproducible, thus promoting FAIR data streams of high quality.

Software and data availability

The System for Automated Quality Control (SaQC) is available both on the Python Package Index (PyPI) and through its Git repository (https://git.ufz.de/rdm-software/SaQC). It is an open source under the GNU General Public License, version 3. At the time of writing, SaQC supports Python > 3.8. The computations in this publication were carried out using SaQC version 2.4.1 and Python 3.11.3. Various tutorials to help the user in getting started can be found in the SaQC online documentation (https://rdm-software.pages.ufz.de/saqc/index.html). A persistent version of SaQC 2.4.1 can be found on Zenodo (https://doi.org/10.5281/zenodo.8092184). Juypter notebooks to reproduce the two applications presented in Section 3, including the necessary data and configuration files, are also on Zenodo (https://doi.org/10.5281/zenodo.8092119).

1. Introduction

Climate change and anthropogenic influences are putting the worlds' ecosystems under significant pressure, requiring a holistic quantification of these impacts. Consequently, large-scale monitoring observatories such as NEON (Loescher et al., 2017), eLTER (Mollenhauer et al., 2018) or TERENO (Zacharias et al., 2011) have been set-up to provide crucial data for the long-term monitoring and modeling of environmental systems. These observation networks are continuously expanding in terms of sensor density and geographical coverage. However, the subsequent steady growth in data volumes poses challenges to those IT infrastructures handling these data streams, especially if processing is required to run in real time: For example, when facing

* Corresponding author at: Research Data Management - RDM, Helmholtz Centre for Environmental Research - UFZ, Permoserstraße 15, Leipzig, 04318, Germany.
E-mail addresses: lennart.schmidt@ufz.de (L. Schmidt), david.schaefer@ufz.de (D. Schäfer).
[1] Corresponding software maintainer.

https://doi.org/10.1016/j.envsoft.2023.105809
Received 15 June 2022; Received in revised form 8 August 2023; Accepted 21 August 2023
Available online 30 August 2023
1364-8152/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

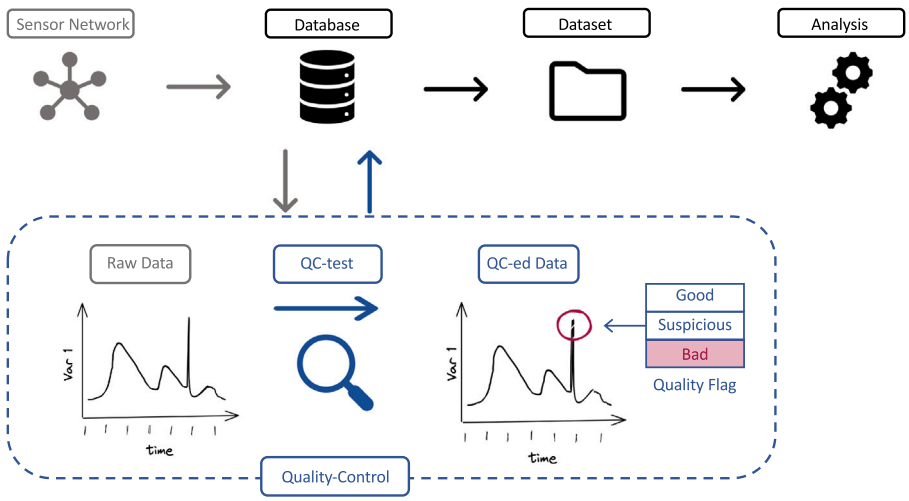

Fig. 1. Illustration of a generalized data flow from sensor to analysis including automated quality control: *Raw data* flow from the sensors into the database and on to the QC-test (here: outlier detection algorithm). The QC-test detects a spike in the data (red circle) and assigns a quality flag, e.g. "Bad", as a measure of its quality. The resulting *QC-ed data*, along with a quality flag for each data point, is sent back to the database, from where it can be published as a dataset and later used for analysis. Raw data are always kept to allow for a potential reprocessing (Campbell et al., 2013).

sudden natural disasters, the transformation of raw data into usable data products has to be timely enough to support live decision-making.

In fact, the only way to enable the real time provisioning of continuously growing data volumes is strict automation (Sturtevant et al., 2021). It is essential to establish automated data pipelines that address one major challenge related to sensor data: the process of quality control (QC), i.e. the separation of erroneous from usable data based on expert knowledge (Koedel et al., 2022; Crystal-Ornelas et al., 2021). The use of data that has not undergone strict quality control can lead to inaccurate model forecasts and have a negative effect on decision-making (Doraiswamy et al., 2000). Errors in environmental data can arise from the malfunctioning of sensors, technical failures during data transmission or the influence of environmental conditions (e.g. fouling sensors, obstructions, freezing etc.) (Gandin, 1988). Manual quality control from visual inspection of the data is laborious, introduces bias, requires expert-knowledge that can neither be reproduced nor transferred and is not available for real-time provisioning (Fiebrich et al., 2010; Jones et al., 2018). Fully automated QC-pipelines, on the other hand can be reproduced and enable the efficient handling of large data volumes, while allowing for unlimited test specifications that can be adapted over time (WMO, 2018). Fig. 1 provides a schematic illustration of the typical data flows in an automated QC-pipeline and introduces two of its core elements: *QC-tests* and *quality flags*.

Fig. 1 also illustrates how automated QC-pipelines are the foundation for the resulting datasets to be Findable, Accessible, Interoperable and Reusable (FAIR, GoFair, 2021). The FAIR-principles request that "[...] optimal care is taken to enable users to determine the 'usefulness' (for their purpose) of the data and other research objects they find, which includes rich, machine readable provenance" (GoFair, 2021). In an automated pipeline with no manual interaction, all processing steps from the raw sensor data to the dataset can be recorded, i.e. they can be traced. This information can then be made available to the user, along with quality information and access to the raw data, enabling the datasets to be "[...] replicated and/or combined in different settings", which is the ultimate goal of the FAIR-principles (GoFair, 2021).

1.1. Existing software for automated quality control

There are documented set-ups of automated QC-pipelines in the environmental community, mostly by large, renowned research infrastructures such as NEON (Taylor and Loescher, 2012), ICOS (Vitale et al., 2020) or IAGOS (Petzold et al., 2015). Usually, these infrastructures develop QA/QC protocols to be fulfilled by the members of the respective network. These protocols require certain algorithm set-ups but do not necessarily give guidance in terms of software implementation. In fact, implementation is down to the members who might be domain-experts with limited technical expertise. This can be a challenge for members and can result in disadvantages with respect to data quality and standardization, requiring additional harmonization steps at the network level (Sturtevant et al., 2021). Similarly, small observation networks and single research institutes are challenged by the software implementation of their QC-workflows.

These custom implementations for automated QC-workflows face the following risks, commonly as a result of limited resources for software development: They might be specific to a particular application and hard-coded, making them less flexible to new functionalities and requirements. Versioning and the ability to reproduce QC-workflows might not be guaranteed, leading to different versions of QC-ed datasets that are published over time without specific notice and limited transparency. At the user interface, programming knowledge might be required in order to parameterize or make changes to the QC-tests, which cannot be presumed to be available. To relieve the sensor network operators from custom software development, the environmental science community needs cost-effective, extendable and user-friendly software programs that facilitate the process of setting up a versioned QC-workflow for any kind of environmental sensor network.

There are few non-proprietary programs for setting up QC-workflows, including the following: (1) *ODM Tools Python* (Horsburgh et al., 2015) is a Python-based program that enables users to query, visualize, and edit time series observations stored in, and thus limited to, an Observations Data Model (ODM) database. The outstanding feature of the tool is its GUI for manual QC that keeps provenance of all manual edits. (2) The *Great Expectations Framework* (Gong and Campbell, 2022) is available as a Python package and provides the user with various so-called "expectations", i.e. tests to check the validity of a given batch of data. It does not provide mechanisms to assign and handle flags at the data point level. (3) The *GCE Data Toolbox* (Sheldon, 2008) requires Matlab (subject to fees) and provides a set of processing functionalities through both a graphical and programming interface. Its set of included QC-tests is limited but enables custom routines to be integrated, while keeping provenance of all operations. (4) *pyhydroqc* is a Python package for automated detection and correction of anomalies using machine learning algorithms (Jones et al., 2022). It provides a modular and extendable workflow structure but is specific to the aquatic domain and model training might be challenging for non-experienced users. Proprietary solutions commonly come as part of an

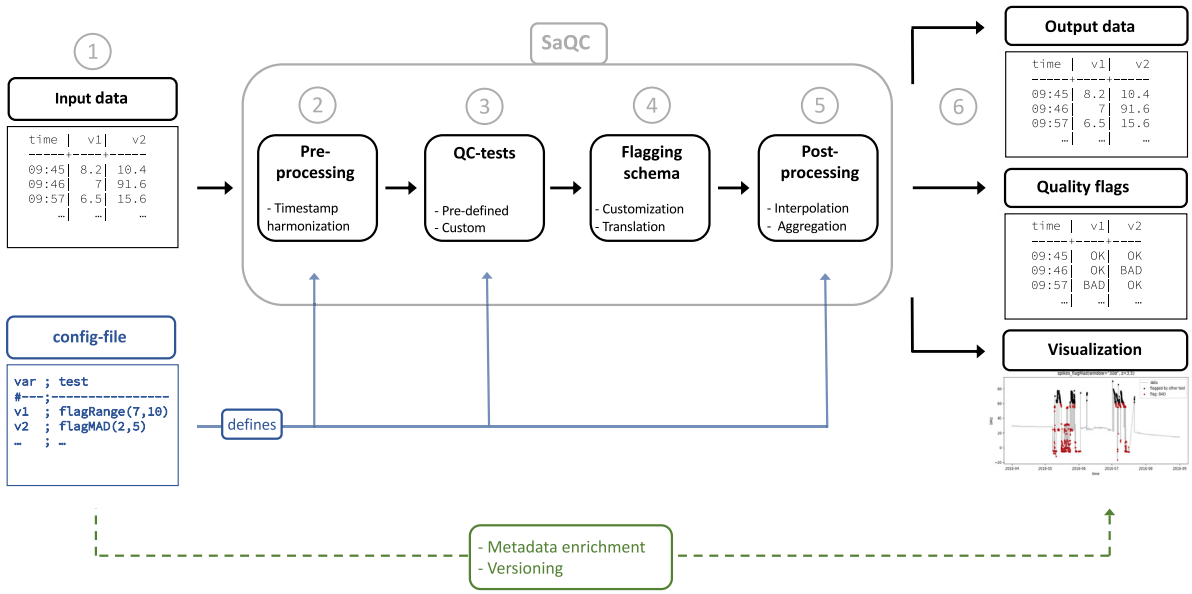

Fig. 2. General QC-workflow using SaQC: Input data and a config-file are sent to SaQC which returns quality-controlled output data, along with quality flags and visualizations of the QC-results. The colors in the figure represent four categories: data flows (black), configuration (blue), metadata flows (green) and annotations (gray). The numbers correspond to the workflow elements as described in the text.

entire data management and alert system. Thus, its implementation within custom infrastructures can be cumbersome. Additionally, these data management systems are typically designed for a specific domain, e.g. Kisters WISKI or Aquarius for the water sector.

The above programs and toolboxes provide important functionality such as versatile GUIs, provenance or extendability. However, each program is subject to its individual limitations regarding the flexibility of input/output structures (option 1), QC-tests (options 2, 3 and 4), affordability (option 3), universal application (option 4), or usability in arbitrary software environments (proprietary solutions).

1.2. System for automated quality control

We present the **System for Automated Quality Control (SaQC)** that aims at overcoming the limitations of existing solutions to facilitate the implementation of standardized QC-workflows for environmental time series data from any sensor. In its development, we aimed for SaQC to meet the following software requirements:

- **Universal:** flexible input/output data structures and flagging schemes; focus on domain-agnostic QC-tests
- **Equipped:** able to provide a comprehensive set of generic pre-/post-processing methods and QC-tests
- **Extendable:** enables the integration of custom processing methods and QC-tests
- **Traceable:** keeps track of processing and flagging operations along the pipeline
- **Reproducible:** enables the versioning of QC-workflows
- **Accessible:** configuration using either a graphical or a low-code user interface
- **Open Source:** available to anyone free-of-charge

This paper provides a detailed software description of SaQC and its features in Section 2. Two applications in Section 3 provide an in-depth explanation of how SaQC can be used in practical applications. These applications present some of its basic and advanced functionalities and evaluate the performance of SaQC. To conclude, we discuss current limitations of the software as well as future development and research directions.

2. Materials and methods

2.1. Software description

SaQC is a free and Open Source software package that aims to facilitate the implementation of standardized QC-workflows for environmental time series data from any sensor. It addresses domain scientists and data managers with limited IT expertise. It can be controlled either through a command line application, a web-based Configuration App or a Python API. All interfaces were designed with the aim of accommodating the exploratory nature of quality control by offering a set of QC tests through a configuration system that is intended to be flexible but with a simple syntax. Besides its QC functionalities, it has traceable routines for data pre- and post-processing as well as the handling and translation of quality flags. Thus, SaQC can be used as a general data processing tool to turn raw sensor data into FAIR secondary data products of controlled quality. The code base was developed with the intention of facilitating the automation of SaQC as part of a data pipeline from a sensor to a data product: The parameterization of QC-tests is technically separated from the rest of the code so that the configuration of QC-workflows can be edited without changing the operational core code. The automated execution of SaQC from a command line or from Python can be carried out as a modular part of various software architectures using common automation tools such as the Cron Daemon (Reznick, 1993) or Apache Airflow (Foundation, 2014).

Based on the generalized data flow from sensor to analysis shown in Fig. 1, Fig. 2 provides a detailed presentation of the QC-component of such a data flow when using SaQC. Below, this SaQC-based QC-workflow is explained in chronological order, with its elements corresponding to the numbers (1)–(6) as indicated in the figure. In this way, the components of SaQC and their contribution to meet the *requirements of QC software* (Section 1) are explained in more detail.

(1) Input data can be any time series data supplied as .csv or .parquet-files when using SaQC as a command line application, in any tabular structure when using the Python API. Using a text-based configuration scheme that requires no knowledge of programming, users can then parameterize existing processing routines as well as QC-tests in a *config-file*. Furthermore, users can write custom routines and tests using a simplified, Python-like syntax. **(2)** The config-file also

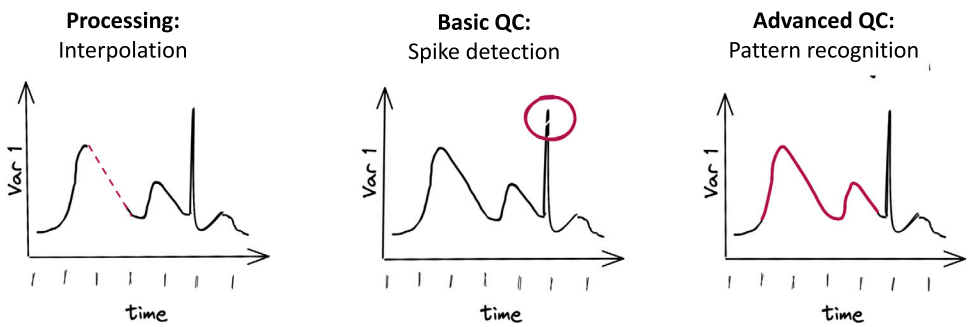

Fig. 3. Visualization of the three application categories: (1) processing, (2) basic QC-test, and (3) advanced QC-tests (from left to right).

defines the pre-processing routines that might have to be executed prior to the QC-test, e.g. timestamp harmonization.

(3) The selection of pre-defined QC-tests aims to be applicable across domains and ranges from standard tests such as the detection of constant values or spikes to more advanced methods like pattern recognition and multivariate tests. Users can also implement entirely new QC-test functionalities in the config-file or as Python code.

(4) Depending on the results of the tests, each data point is assigned a flag following a user-defined flagging scheme that defines the number and hierarchy of the flags, e.g. *Good*, *Suspicious* and *Bad*. SaQC allows entirely user-defined flagging schemes with the functionality of translating them from one to another.

(5) If desired, post-processing routines such as interpolation or aggregation over time can be applied to the newly quality-controlled data.

(6) The processed data, along with the assigned flags, are then returned as .csv or .parquet-files (command line application) or any other user-defined format (Python). Adding to the file output, visualization functionalities allow the user to investigate the effect of different test parameterizations on the QC-results.

Across the whole workflow, SaQC ensures traceability and reproducibility through metadata enrichment and the versioning of pipelines.

2.2. Function overview

SaQC ships with processing capabilities as well as various QC-tests, providing a total of 32 functions. The development team plans to continuously add QC tests. Fig. 3 aims to give a representation of how these functions are further categorized into three categories: **(1) Processing functions** serve as either the preliminary (pre-processing) or finalizing (post-processing) step to the actual QC operations. They include timestamp harmonization as well as the smoothing and transformation of data. **(2) Basic QC-tests** can be parameterized relatively quickly and detect periods of too little variability (constant) and discontinuous data (breaks, jumps, outliers). **(3) Advanced QC-tests** cover more complex algorithmic operations such as the detection of noise, change points or user-defined patterns. Advanced QC-tests require elevated parameterization effort and, potentially, training of statistical models.

Table 1 lists all functions as included in SaQC.

2.3. Configuring SaQC for application

SaQC can be controlled either through a **command line application**, a web-based **Configuration App** or a **Python API**. As a **command line application**, SaQC is controlled by a config-file (.csv) listing the variables of the dataset and both the processing routines and the QC-tests along with the respective parameters that are to be executed. The content of this kind of configuration could look similar to this:

```
varname     ; test
------------;-----------------------------------
# pre-processing
SM2         ; align(freq="15Min", method="nshift")
# basic QC
SM2         ; flagRange(min=10, max=60)
SM2         ; flagMAD(window="30D", z=3.5)
# plotting intermediate results to file
SM2;        ; plot(path=PLOTPATH1)
# post-processing
SM2         ; interpolate(limit=2)
# plotting final results
SM2         ; plot(path=PLOTPATH2)
```

Lines starting with a # are comments. For a variable called SM2, a timestamp shift to a regular interval of 15 min is performed during preprocessing. This is followed by a range test that flags all values outside the range of [10,60] and a spike test based on the median absolute deviation (MAD, see Iglewicz and Hoaglin, 1993). The results after each or multiple computation steps can be plotted to a user-defined file path (here: PLOTPLATH1). Next, data gaps of up to two data points are filled using linear interpolation. The final result is plotted to PLOTPLATH2. In fact, this is the config-file as used for application 1 in Section 3.1.

SaQC can be run from the command line by providing the file paths to the config-file, the input dataset and the desired output file to contain both processed data and flags (see Box I).

The same set-up can be achieved directly with a Python script by using SaQC's **Python API**, which is explained in more detail in the software documentation.

For users without a programming background, there is an alternative way to derive a suitable config-file using the web-based **SaQC Configuration App** (https://webapp.ufz.de/saqc-config-app/), displayed in Fig. 4. It offers a graphical user interface for users to upload their own data, parameterize the available preprocessing and QC-test functions and visualize the results. At the end, users can obtain the respective config-file to use in their own QC-pipeline.

2.4. Implementing custom functions

Users can implement custom functions in the config-file. The flagGeneric-function can be used to build function expressions using a simplified syntax and a set of operators as listed in Table 2. A simple config-file to flag all values greater than 30 of a variable x would be:

```
varname ; test
#-------;------------------------
x       ; flagGeneric(func=x < 30)
```

Custom functions can also be used to cross-reference variables, i.e. to flag x where two other variables, y and z, both exceed 30 and 50, respectively:

Table 1
Processing and QC functions of SaQC. Grouped into functional groups and with a description of each functions' objective. For legibility, the table only lists the most relevant processing and QC-functions. Various specific helper functions are listed in the software documentation.

Group	Function name	Objective
Processing	*Processing steps prior to (pre-processing) or after QC-testing (post-processing)*	
Harmonization	align	Align data points and flags to a regular frequency grid, using interpolation or shifting
	resample	Resample data points and flags to a regular frequency grid
	interpolate	Interpolate data points and flags to a regular frequency grid
Smoothing	rolling	Process data by rolling window calculations
	fitPolynomial	Smooth data by window-wise fitting of polynomials
Transformation	transform	Transform data by applying custom functions on windows of variable size
	processGeneric	Generate/process data with custom functions
Flag projection	concatFlags	Append flags and flagging histories from one time series to another
Basic QC-tests	*Low algorithmic complexity and parameterization effort*	
Constants	flagConstants	Flag data where they are strictly constant
	flagByVariance	Flag data windows with a variance below a given threshold
Breaks	flagMissing	Flag missing data
	flagIsolated	Flag groups of values isolated by missing or flagged data
	flagJumps	Flag data where the window-wise arithmetic mean of its values changes significantly
Outliers	flagRange	Flag values that exceed a given range
	flagOffset	Flag values using customizable deterministic outlier detection
	flagByStray	Flag data using the STRAY algorithm
	flagMAD	Flag outliers using modified Z-score outlier detection
	flagByGrubbs	Flag outliers using the Grubbs algorithm
	flagMVScores	Flag outliers using multivariate k-nearest neighbor aggregation
Manual flagging	flagManual	Transfer pre-existing (manual) flags from auxiliary files
Advanced QC-tests	*Higher algorithmic complexity and parameterization effort*	
Noise	flagByStatLowPass	Flag data based on window-wise tests in the frequency domain
Changepoints	flagChangePoints	Flag data points that represent a system state transition between two adjacent rolling windows
	flagRegimeAnomaly	Flag data windows that behave abnormal based on agglomerative clustering
Drift	correctDrift	Remove drift component from data by fitting a linear, exponential or custom drift model
	flagDriftFromReference	Flag data where it deviates from a reference dataset
	flagDriftFromNorm	Flag data windows that gets assigned to a minority group by agglomerative clustering
Pattern recognition	flagPatternByDTW	Flag data windows based on their distance to a given pattern using dynamic time warping
Custom functions	flagGeneric	Flag data based on application of user-defined functions
Machine learning	trainModel	Train an ensemble of tree-based boosting and bagging models with an Autotuner (MLjar)
	modelPredict	Predict data with a trained regressor
	modelFlag	Flag data with a trained classifier
	modelImpute	Impute data with a trained regressor

```
saqc -c path_to_configuration.csv -d path_to_data.csv -o path_to_output.csv
```

Box I.

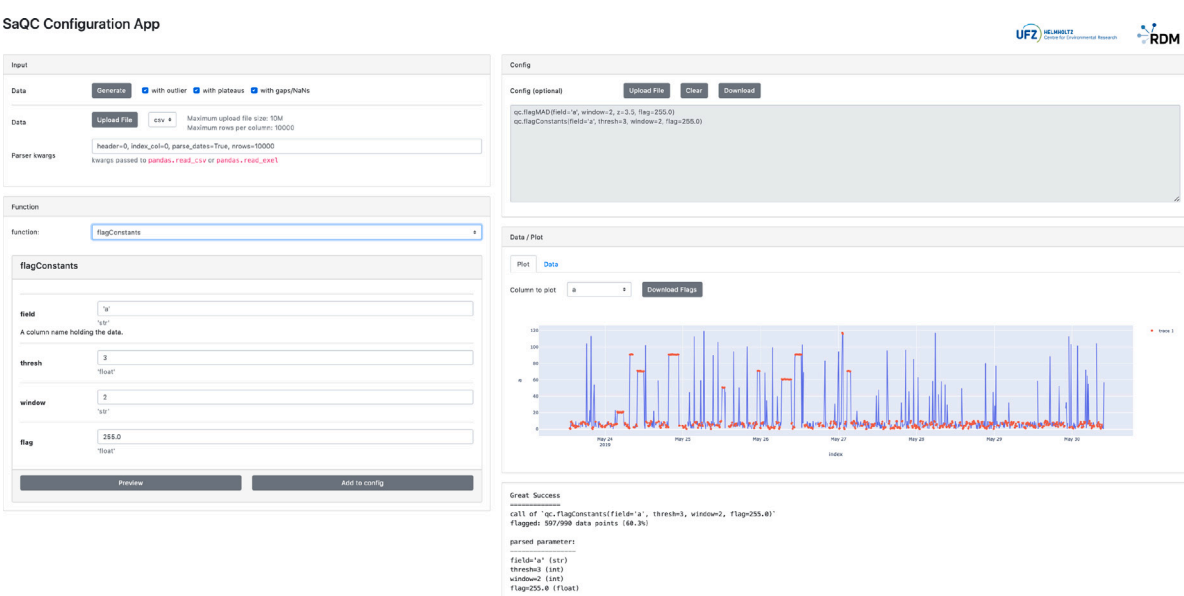

Fig. 4. Graphical User Interface of the SaQC Configuration App.

Table 2
Overview of operators for the construction of custom functions in the config-file.

Purpose	Operator	Description
Comparison	`==, !=, <, > <=, >=`	equal, not equal, greater, less than, greater or equal, less than or equal
Logical	`&, | , ^, ~`	binary AND, OR, XOR, complement
Arithmetic	`+ , -, *, /, **, %`	addition, subtraction, multiplication, division, exponentiation, modulus
Extremes	`min, max`	minimum, maximum
Mathematics	`exp, log`	exponential, logarithm
Statistics	`mean, median, std, var, cv`	mean, median, standard deviation, variance, coefficient of variation
Standardization	`iqsScore, madScore, zScore`	inter quartile range, mean absolute deviation, standard deviation
Miscellaneous	`abs, count, diff, sum`	absolute value, number of values, first discrete difference, summation

```
x      ; flagGeneric(field=["y", "z"], func=(y > 30) & (z < 50))
```

In addition to the above operators, there is a set of mathematical expressions to compute various moments of the data such as minimum, maximum, mean or standard deviation (for a complete list please refer to Table 2) . As an example, the following expression flags `x` where it exceeds two standard deviations of `z`:

```
x       ; flagGeneric(field=["x", "z"], func=x > std(z) * 2)
```

Using the Python API instead of the config-file, arbitrary functions that handle the given variables can be passed to `flagGeneric`. For more complex needs, SaQC's modular structure also allows custom functions to be integrated that adhere to the global function interface, into the program and data flow.

2.5. Flag handling inside SaQC

One of the key development goals of SaQC was that all processing and QC-functionalities should accommodate explicit, user-defined flag handling. When chaining multiple processing steps and QC tests in practical application, flag handling raises challenges that can become complex. By means of five key challenges, the following section illustrates the potential complexity and how it can be addressed when using SaQC.

2.5.1. Filtering of flags between QC-tests

Some QC-tests might require the data to be filtered according to the flags from preceding QC-tests. The example config-file below presents such a case where a range-test flags all values from sensor data `SM2` outside the range of [10,60] as BAD. This might lead to remaining, unflagged data points that are isolated, i.e. surrounded by flagged data only. A user might want to run a `flagIsolated`-test in the next step to flag these isolated values as well. However, this second test will only work if it actually only sees the remaining data, i.e. by excluding all data points that were flagged as BAD before. This can be achieved by using the filter keyword `dfilter` which can be applied to any of the QC-tests.

```
varname      ; test
------------;-----------------------------------
SM2          ; flagRange(min=10, max=60)
SM2          ; flagIsolated(dfilter=BAD)
```

2.5.2. Conditioning of QC-tests based on previous flags

The execution of one QC-test might have to be conditioned on the results of the preceding QC-tests. In the example configuration below, sensor data `SM2` might be affected either by a drop in sensor battery voltage `BattV` or due to maintenance work on the power supply. Thus, both a range test and manual flagging (based on an auxiliary file of maintenance events) are executed on `BattV`. After that, a custom function `flagGeneric` is conditioned on the two preceding tests by making use of the option to chain expressions via boolean operators (here: boolean OR, |): If one or both of the tests assigned a flag to `BattV`, the sensor data `SM2` is flagged accordingly.

Table 3
Illustration of a flagging history for sensor data *SM2* after running three QC-tests. Each flag can be traced back to the respective QC-test of origin by storing flags in columns.

time	SM2	flagRange	flagMAD	flagByStray
2016-04-01 00:05:00	30.0	BAD	GOOD	GOOD
2016-04-01 00:20:00	32.0	GOOD	GOOD	GOOD
2016-04-01 00:35:00	44.0	GOOD	BAD	BAD
...				

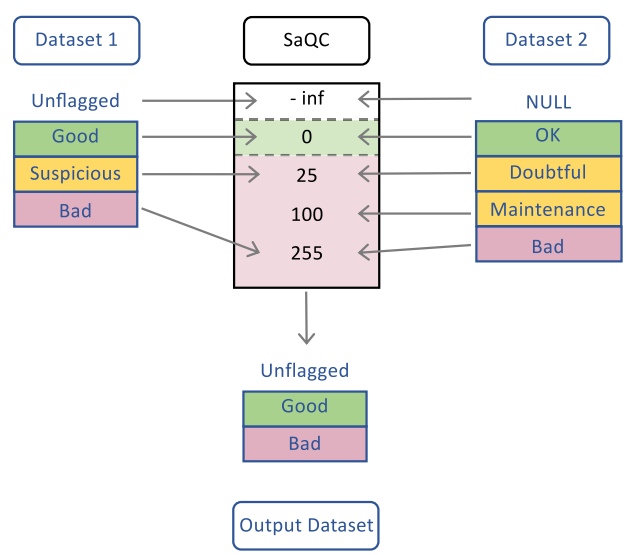

Fig. 5. Top row: Illustration of the mapping of two datasets' different flagging schemes (left & right) onto the internal numerical encoding of SaQC (center). The internal encoding is a real-valued vector inside [-inf, 255.0]. Flags of the same hierarchy (e.g. "Good" and "OK") can be mapped onto the same numerical value, while flags exclusive to a dataset get assigned individual values (e.g. "Maintenance"). Light colors indicate the internal reclassification that yields the output scheme (bottom): All flags that are not $-inf$ (no flag set) or 0 ("OK"/"Good") are reclassified to "Bad". Please note that the exact choice of numerical values is user-defined.

```
varname      ; test
------------;---------------------------------------
BattV        ; flagRange(min=.5, label="voltage drop")
BattV        ; flagManual(mdata="maintData", label="maintenance")
SM2          ; flagGeneric(func=isFlagged(BattV, "voltage drop")
```

2.5.3. Provenance of the QC-tests for each flag

For each flag, information on the QC-test that produced it has to be traceable after the QC-process has finished. In SaQC, this is achieved by storing and returning the entire flagging history, i.e. the results of all QC-tests for each data point (see Table 3). This also allows a final aggregation of all assigned flags into a numerical score of data quality as presented in e.g. Kaffashzadeh et al. (2019).

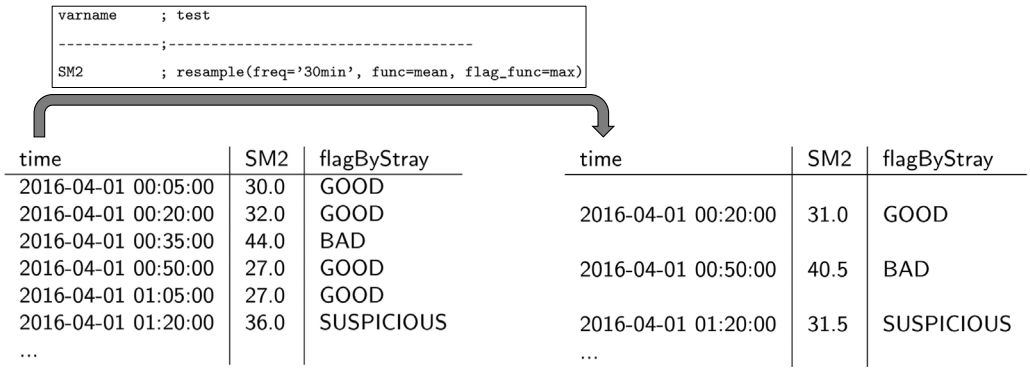

Fig. 6. Illustration of resampling both data values and flags on sensor data *SM2*. Top: The respective config-file to define the resampling interval, the data aggregation function *func* and the flag aggregation function *flag_func*. Left: Original data and flags at 15 min-resolution. Right: Resulting data and flags at 30 min-resolution.

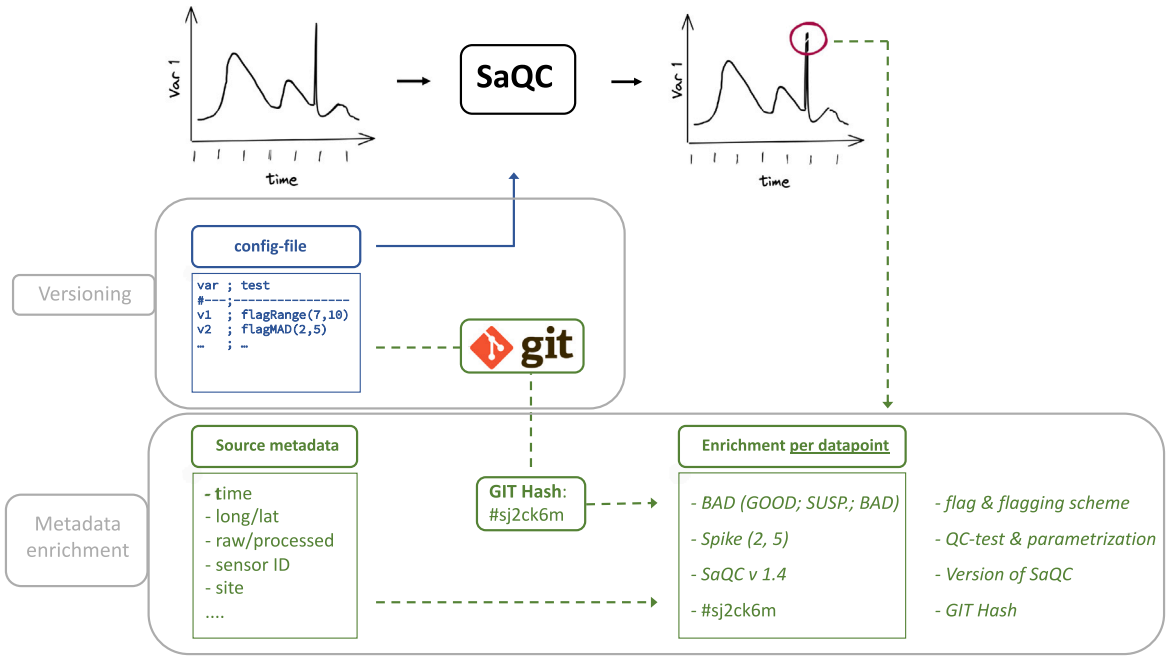

Fig. 7. Schematic illustration of metadata enrichment (traceability) and versioning (reproducibility) when using SaQC. The red circle marks a data point identified as a spike by a QC-test. The other colors distinguish among four categories: Data flows (black), configuration (blue), metadata flows (green) and annotations (gray).

2.5.4. Translation of flagging schemes

Datasets from separate sources may need to be merged for further processing. For example, soil moisture data from a local observatory might have to be merged with precipitation data from a national weather service to be published as a joint data product that includes data quality information. If these separate datasets have already undergone prior QC, they can exhibit dissimilar flag levels, i.e. be based on different flagging schemes (compare Fig. 5, top). To merge the data and quality information of both datasets, their flagging schemes need to be mapped onto each other while preserving flag hierarchies. Finally, the user might decide to harmonize the quality information by defining yet another flagging scheme for the final output (Fig. 5, bottom).

SaQC addresses this application by representing a datasets' flags as a series of floating point values internally (Fig. 5, top center). Generally, the higher the numeric value, the "worse" the flag. This representation enables the encoding of a wide range of potential flagging schemes including their inherent hierarchy. The numerical encoding also allows comparisons and aggregation of flags using mathematical operators and functions like `<`, `>`, `min`, `max` etc.

2.5.5. Keeping track of flags during resampling

The internal flag encoding is also used when resampling datasets. Fig. 6 illustrates the resampling of sensor data SM2 from 15-min temporal resolution to 30-min resolution. As indicated by the `func`-parameter, the mean of each pair of data values at 15 min-resolution is calculated to return one value for the respective 30 min-interval. As for the flags that were already assigned at 15 min-resolution, the user can define a flag aggregation function, `flag_func`, to determine which flag should be assigned to the 30 min-interval. These can be mathematical functions to be run on the internal numerical flag representation. Here, `flag_func=max` indicates that the flag of worst data quality is assigned to the 30 min-interval. Internally, the original data structure is kept, so that additional flagging results on the resampled data can be projected back onto the original data.

2.6. Enabling traceable and reproducible workflows

SaQC supports metadata enrichment for each data point and versioning of the entire pipeline including all parameterizations through version control systems (e.g. Git). As such, every single quality flag is not only traceable, but can be reproduced if necessary. Fig. 7 is a

Fig. 8. Left: Birds-eye view of the forest ecosystem observatory Hohes Holz. Right: Measurement set-up of multiple soil moisture sensors at different soil depths as employed in application 1. Image source: UFZ.

variation of the SaQC-workflow previously depicted in Fig. 2 with a focus on the metadata component (green color). Fig. 7 illustrates how metadata enrichment and versioning can be achieved in SaQC-based workflows: The input data comes with source metadata, e.g. sensor location, ID or data author. As it runs through the SaQC-workflow, and as such through the QC-tests, metadata are collected and merged with the source metadata. For instance, in the case of a data point identified as a spike (red circle in Fig. 7), this includes the quality flag as well as information on the flagging scheme that it refers to. The QC-test that assigned the flag (here: spike-test) is also written, which becomes relevant when multiple QC-tests are carried out on the same variable. To account for different implementations across software versions, the SaQC version is also noted. In order to keep track of the version of the entire workflow, the config-file has to be versioned through Git and the respective Git Commit Hash can be written into the metadata. With this, a user receiving any dataset that was QC-ed using SaQC is able to **(1) trace** and **(2) reproduce** every step of the pipeline.

3. Applications

This section presents applications of SaQC in actual applications at the Helmholtz-Centre for Environmental Research (UFZ), more specifically at two observatories that are part of the TERENO Bode Hydrological Observatory which is located in the Harz region in central Germany (Wollschläger et al., 2017). For the first application at the Hohes Holz observatory (Rebmann et al., 2017), we employed some of the functionalities listed as *Processing* and *Basic QC-test*. This application was also used to perform a quantitative evaluation of the performance of SaQC with regard to the QC results and computational demands. The second application at the lower Bode river (Huang et al., 2022) served to illustrate the potential complexity of QC-workflows for increasingly used surface water quality sensors (Rode et al., 2016). This application illustrates how multiple routines build on top of each other when using *Advanced QC-tests*.

3.1. Application 1: Basic quality control of soil moisture data

The Hohes Holz observatory is a patch of mixed beech forest where meteorological, hydrological and ecological variables are observed at high spatial and temporal resolution (see Fig. 8). The goal is to improve the scientific understanding of carbon and water fluxes between the ecosystem and the atmosphere under the influence of environmental changes. One of the observed variables is soil moisture content which is measured by 180 individual sensors. In this application, the quality control of a soil moisture dataset from one sensor was performed in four steps. The dataset contained the variables: `time`, `battery` (battery voltage of the data acquisition platine) and soil moisture `SM2` in vol.%. The following description covers the actual config-file from Section 2.3, that is also included in Appendix A.1, line for line. Fig. 9 depicts all steps of the workflow, its top row showing the raw data of sensor `SM2` before quality control. It shows oscillations beyond a plausible value range that could not be explained by domain knowledge and were assumed to be an error.

Time stamp harmonization through resampling: As a result of irregular data acquisition frequency of the wireless sensor network, the timestamps of the measurements were not equidistant. Further processing steps required the datapoints to be on a regular timestamp grid. Thus, they were shifted to the closest timestamp of a 15 min-grid, thereby also eliminating any potential duplicates. The corresponding line in the config-file can be found below.

```
varname ; test
#-------;-----------------------------------
SM2     ; align(freq="15Min", method="nshift")
```

Range test: a range test to exclude values below/above a physically meaningful threshold was then performed (here: [10, 60], vol.% soil moisture).

```
SM2     ; flagRange(min=10, max=60)
```

MAD outlier test: Following that, a Modified Z-Score (MAD) outlier detection test, as proposed by Iglewicz and Hoaglin (1993), was performed to detect and flag spikes in the data. The resulting flags of steps 2 and 3 are depicted in Fig. 9 (center).

```
SM2     ; flagMAD(window="30d", z=3.5)
```

Linear interpolation of missing values: As a final post-processing step, the missing values that resulted from identifying error values during the flagging beforehand are filled using linear interpolation (up to a data gap of two data points, see below). The final, clean results are displayed in Fig. 9 (bottom).

```
SM2      ; interpolate(limit=2)
```

3.2. Application 1: Performance evaluation

For evaluation of the results, we retrieved reference flags from a semi-manual QC-workflow that was operational before the use of SaQC on this data. The reference workflow consisted of two automated

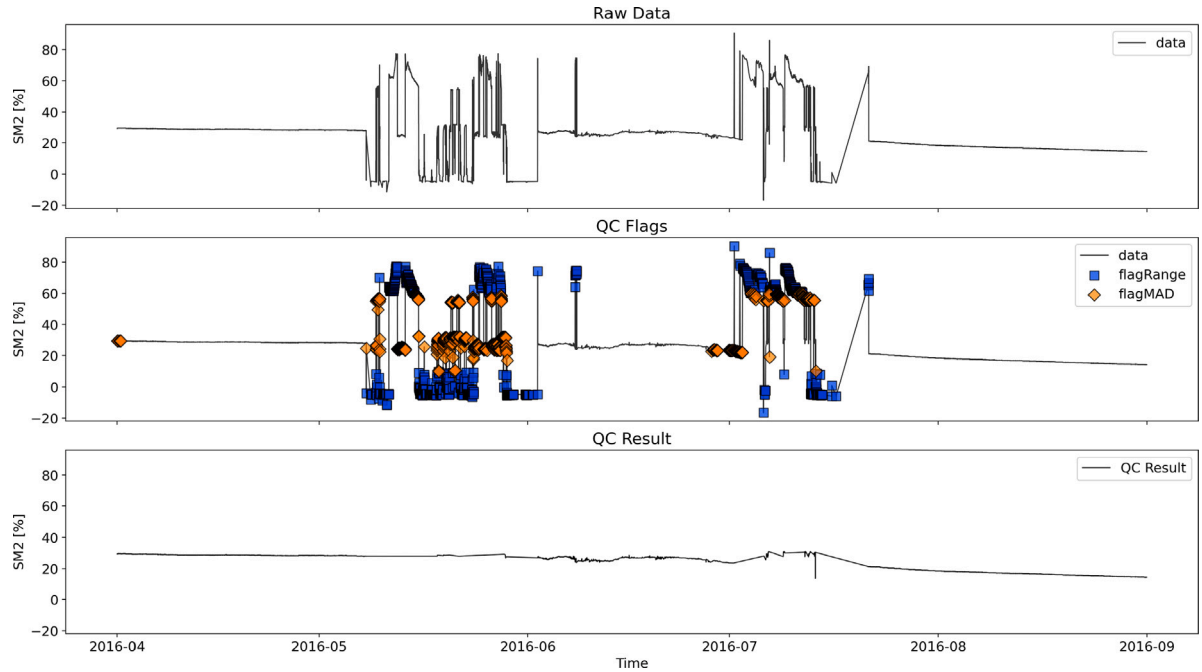

Fig. 9. Top: Raw soil moisture data *SM2* from one sensor of the distributed soil moisture sensor network at the research site 'Hohes Holz'. Centre: Quality flags of the variable *SM2* as a result of flagRange and flagMAD. Bottom: Final result after the QC process.

QC-tests, a range test and the detection of outliers, that were post-corrected by visual inspection of a domain expert. The comparison enabled a quantitative performance evaluation of the configuration of SaQC used in this application. One should bear in mind that *quantitative* is not equivalent to *objective*. Automated QC (if parameterized manually as in this example), and manual QC both carry varying degrees of subjectivity. One option for a ground truth that is as objective as possible is to perform collaborative QC by a group of people as done in Jones et al. (2018).

A comparison of the reference flags and the ones by SaQC is shown in Fig. 10. It can be seen that both workflows flagged oscillations around a value of about 30%. The flagging patterns were similar, both dominated by a range test that flags the majority of data points. The periods in between were covered by outlier tests (BAD:MAD, BAD:Offset) or manual flags (BAD:Manual). The outlier-test of the SaQC-workflow (MAD) was more sensitive than the offset-test in the reference, thus also covering manual flags of the reference. The major difference between the two can be seen in the period 06/2016 to 07/2016: In the reference, the domain expert considered this period to contain errors, possibly because it is between two periods of major errors. These data points were not flagged in the SaQC-workflow, and are reflected in the confusion matrix of the final flags (Table 4), i.e. BAD if any test set a BAD-flag: Despite similar flagging patterns, the SaQC-workflow only picked up 57% of the flags set by the reference (*Recall* or *Sensitivity*). On the other hand, there were only 8 false positives, i.e. data points that were flagged by SaQC but not by the reference (*Precision*). When we omitted the data from 06/2016 from the analysis, recall rose to 77% (Table 5). The remaining flags that SaQC did not set similar to the reference can be seen as isolated data points in 05/2016 and 07/2016 in Fig. 10 (top). These isolated points could be identified using e.g. the `flagIsolated`-test in SaQC but were considered to be close enough to the error-free data during parameterization.

Overall, the comparison shows that SaQC can be used to reproduce the flagging pattern of a manual expert to an extent where differences can be attributed to the subjectivity of parameterization rather than to technical limitations. The configuration of SaQC used in this application performed well enough to replace manual efforts but can still be improved.

Table 4
Confusion matrix of the final reference flags (rows) and those derived using SaQC (columns) in application 1. For both, a data point was assigned a "BAD"-flag if one or multiple QC-tests/manual inspection identified an error value.

		SaQC		
		BAD	GOOD	All
Reference	BAD	2710	2038	4748
	GOOD	8	9933	9941
	All	2718	11971	14689
	Precision = 1.00			
	Recall = 0.57			

Table 5
Confusion matrix of the final reference flags (rows) and those derived using SaQC (columns) in application 1 when excluding the month of June from the analysis. A data point was assigned a "BAD"-flag if one or multiple QC-tests/manual inspection identified the value as an error.

		SaQC		
		BAD	GOOD	All
Reference	BAD	2668	779	3447
	GOOD	8	8354	8362
	All	2676	9133	11809
	Precision = 1.00			
	Recall = 0.77			

3.3. Application 1: Runtime

To estimate the runtime of SaQC on different data volumes, we duplicated the raw data from application 1 to obtain 1e3, 1e4, 1e5, 1e6 and 1e7 data points. Fig. 11 shows the results both from running the config-file from application 1 and by only tracking the overhead when loading data (Pandas dataframe) into an SaQC-object in the Python API. The experiment was run on a standard office laptop (Lenovo ThinkPad X390, Intel i7-8565U CPU, 16 GB RAM) using the Python API. Both runtimes increased more or less linearly with the number of data points.

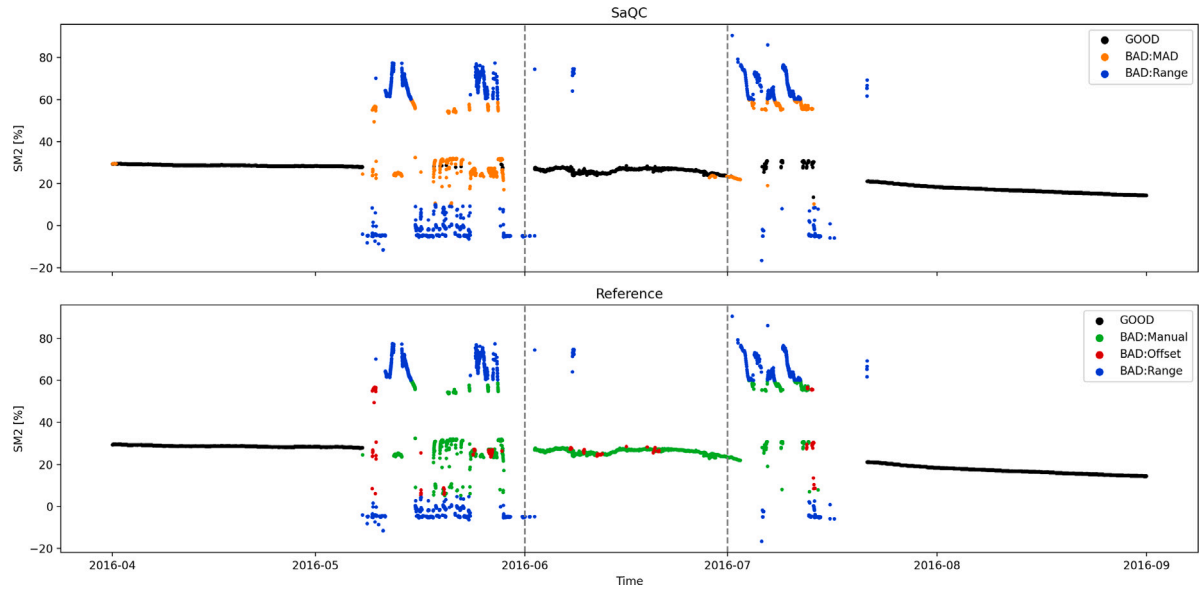

Fig. 10. Time series of flagging results by SaQC (top) and the reference (bottom). As each data point could theoretically be assigned a BAD-flag from each QC-test, plotting order had to be fixed according to the order in the legend (top: background, bottom: front). Both time series contained flags from a range-test, an outlier test (MAD/Offset) and the reference also contained manual labels. Gray bars indicate the start and the end of the month of June which was omitted in parts of the analysis.

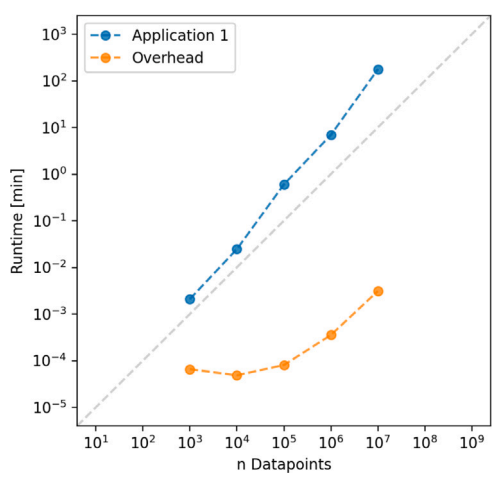

Fig. 11. Results of the runtime experiment on application 1 data. The runtime of SaQC (y-axis) was recorded for different numbers (n) of data points (x-axis). Both the x and y-axis are on a log-scale. Blue represents the runtime when executing the configuration of application 1. Orange represents the runtime, measuring only the computational overhead of loading the data into SaQC. Gray stands for the reference line of a desired linear increase, i.e. a relationship of $y = a + x$.

The maximum case of 1e7 data points corresponds to about 285 years of data from one sensor with a 15 min-resolution. This is equivalent to about 7 years of data from 40 sensors, a data volume that is commonly encountered in practical application on sensor networks. It was processed in 103 min (overhead: 1.1s). Common QC configurations can become more complex than in application 1, requiring more processing time. Still, this analysis shows that the runtime of SaQC can be expected to increase linearly with larger datasets from entire sensor networks.

3.4. Application 2: Advanced quality control of surface water body data

The lower Bode river is the outlet of the TERENO Bode Hydrological Observatory. It is strongly controlled by the Rappbode reservoir, Germany's largest drinking water reservoir. Hydrological variables in the Rappbode/Bode system are recorded to investigate the dynamics of nutrients and dissolved organic carbon (DOC) from the upstream forested areas as well as from the agricultural lowlands (see Fig. 12). For more details on the observatory, please see Rinke et al. (2013) or Wollschläger et al. (2017). At the respective lower Bode gauge station (Stassfurt), the variables `water level`, `water temperature`, and `sac254` (spectral absorption coefficient at 254 nm used as a proxy for DOC) are measured. The station is run by the hydrological state authority (State Agency for Flood Protection and Water Management of Saxony-Anhalt — LHW). These measurements are highly sensitive and require regular maintenance as well as sophisticated quality control, which is performed in four steps. In this section, the first three steps are described for one of the variables, `sac254`, only. In the last step, all three variables are used as covariates for multivariate flagging. This description deals line by line with the actual configuration file needed for the final results (see Appendix A.2). Fig. 13 depicts all steps of the workflow. Its top row shows the raw time series of `sac254`, i.e. before quality control. `sac254` shows undesired outliers and cyclical falls in its data values that could not be explained by domain knowledge and were assumed to be errors.

Flagging of maintenance intervals: As a first step, data recorded during known maintenance operations was flagged, as these operations are likely to affect the measurements. The respective information had been encoded in a separate variable `maint` and used inside the manual flagging function (Fig. 13, second row).

```
varname ; test
#-------;-----------------------------------------------
sac254  ; flagManual(mdata="maint")
```

Time stamp harmonization through resampling: As the sensors do not necessarily measure at the same interval or exact same time, the time stamps of all variables were resampled to an equidistant grid of 15 min-intervals. This was required for subsequent multivariate QC-tests. As the data values themselves had to be adapted accordingly, linear interpolation was performed on the data values where necessary, not touching on actual missing values.

```
sac254   ; align(freq="15min")
```

Drift correction: The variable `sac254` exhibited a short-term drift: Due to biofilm growth and dirt accumulation on the sensor lenses, the

Fig. 12. Left: Birds-eye view of the Rappbode reservoir, as part of hydrological observatory. Right: Measurement site at its outlet, the lower Bode, carrying sensors as employed in application 2. Image source: UFZ.

```
sac254_corr  ; correctDrift(field="sac254", maintenance_field="maint", model="exponential")
```

Box II.

reference value of the measurements increased over time as illustrated in Fig. 13 (third row). Whenever maintenance took place, the sensor was cleaned. Based on expert knowledge, these increases were assumed to follow an exponential growth that started after each maintenance operation and was reset to a new reference value when the subsequent cleaning operation took place. The drift was corrected by subtracting an exponential term that was fitted to the period between the two maintenance intervals. The points in time of maintenance operations were retrieved from the `maint`-column. Due to a change in sensor set-up, it was primarily the first half of the time series that was prone to sensor drift (see Box II).

Multivariate spike detection using kNN and STRAY: It was not only the information from `sac254` but from all three variables (`water level`, `water temperature` and `sac254`) that was used to identify multivariate outliers. The methodology was based on the *oddwater*-algorithm by Talagala et al. (2019). The aim was to use the well-known unsupervised clustering algorithm k-Nearest Neighbors (kNN), which required the variables to be normalized in a first step using z-score transformation. The kNN-algorithm was then used to assign a multivariate outlier score (new variable: `kNN_scores`) to each time step. Fig. 13 (fourth row) gives a visual overview of how, during this dimensionality reduction, multivariate outliers were detected when transformed into one variable. The `kNN_scores` were then processed further using the STRAY-algorithm (Talagala et al., 2021) to automatically define a threshold above which data points of the `kNN_scores` were considered to be actual outliers and flagged as such. As a last step, these flags were projected from the `kNN_scores`-variable back onto the actual data variables. Fig. 13 (fifth row) depicts the results after this projection was performed for `sac254` (see Box III).

The final, clean results are presented in the bottom row of Fig. 13. In general, all major error patterns (major outliers, cyclical drops, drift) were corrected for. A few spikes remain that should be analyzed in more detail by the domain expert and that could be covered by one of the spike tests included in SaQC. However, it is to be noted that the figures shown here only depict snippets of the whole time series from one sensor while the observatory consists of numerous sensors with slightly different error characteristics. As finding the right parameterization for a time snippet of only one individual sensor already requires substantial efforts from the domain experts, finding a robust parameterization for the whole network is always a trade-off between local performance (time series snippets of one sensor) and global performance (the entire network).

4. Discussion

As mentioned in the above applications, the parameterization and evaluation of QC procedures is subjective and bound to be a trade-off between invested time and local versus global performance. Nevertheless, the results suggest that SaQC meets its development goals: Both simple and complex workflows can be set-up to effectively reproduce manual QC for different environmental variables in a reproducible way and without programming knowledge. Scalability to large sensor networks can also be expected.

Although it was designed to be as universal as possible, there are still some technical limitations to the software that are outlined in the following, along with future developments.

4.1. Limitations of the software

As already pointed out, SaQC was designed for “environmental time series data”. This refers to the authors‘ past applications and tests which were conducted on environmental variables, only. Technically, any time series data can be loaded and QC tests were chosen so that they were not limited to any variable or domain. In terms of input file types, these are currently limited to .csv and .parquet if SaQC is used from the command line interface (CLI), whereby netCDF support is intended. If used from the Python API, any file format or database API can be implemented by the user. As for traceability and reproducibility, (1) versioning and (2) metadata enrichment are optional functionalities. If desired, the user is required to (1) host the QC-pipeline on the code versioning platform Git and (2) be able to store additional metadata (SaQC version, flagging scheme, QC-test that was executed etc.) for each data point in the respective data repository. Depending on the system in use, the latter can potentially result in a considerable computational load if the configuration of the pipeline is changed and all historical data points are to be re-processed.

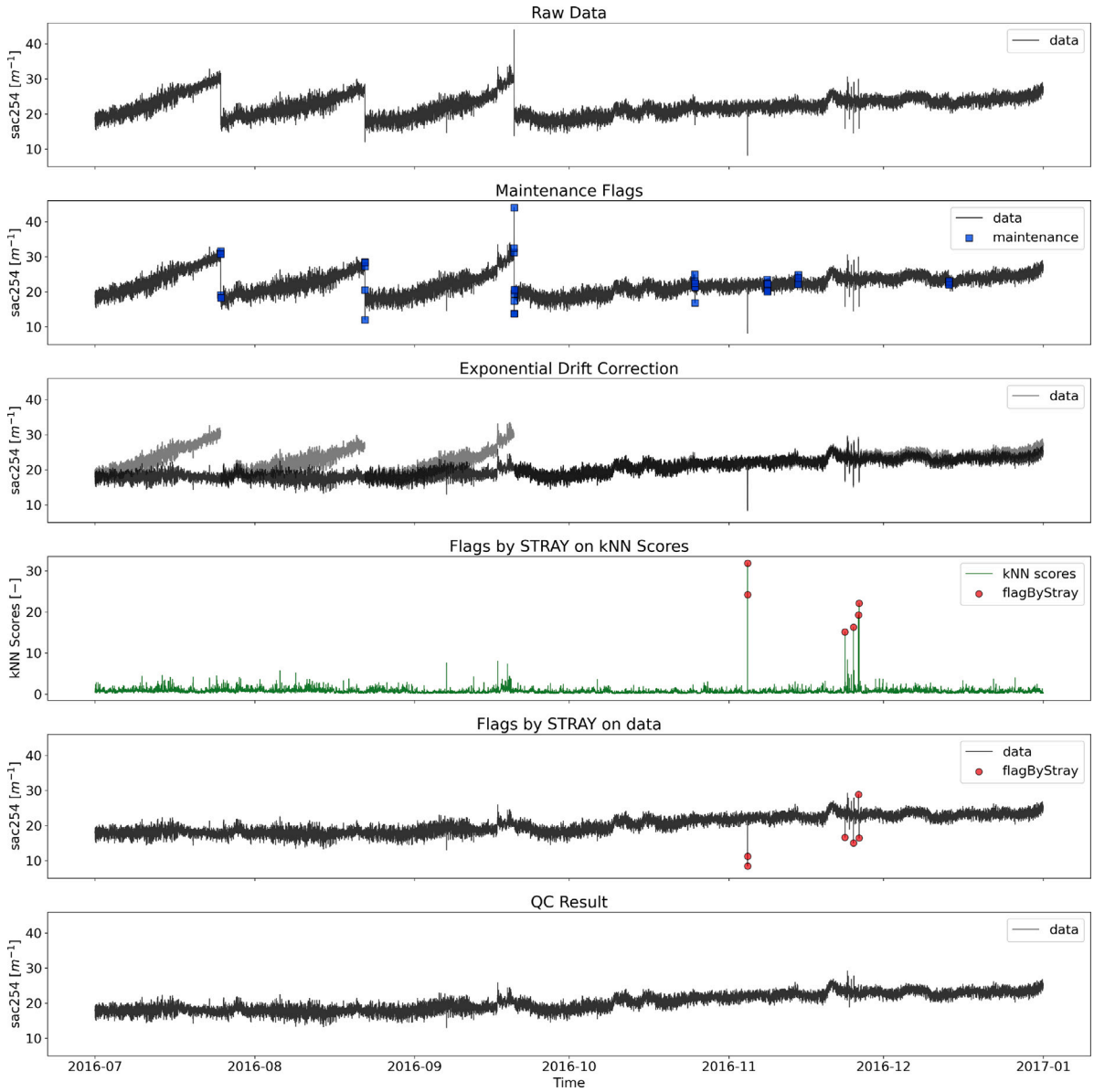

Fig. 13. Data and flags of all QC steps of application 2. From top to bottom: (1) Raw data of the variable *sac254* before quality control. (2) Flags of maintenance operations on *sac254*-data using *flagManual*. (3) Illustration of the drift correction on *sac254*-data. Gray: original data, Black: corrected data. (4) Time series of kNN-scores derived from all three variables (*water level*, *water temperature*, *sac254*) and multivariate outliers that were identified by the STRAY-algorithm. (5) The corresponding multivariate outliers projected onto and visualized for *sac254*, only. (6) Time series of *sac254* after the QC-process, i.e. clean data.

4.2. Future development of the software

The future development of SaQC is intended in different ways: It will be implemented in other real-world environmental monitoring workflows, both at the authors' institutions and beyond. In doing so, the existing set of QC-tests will be continuously extended by both domain-specific and -agnostic tests. A manual parameterization of the QC tests might, however, become unfeasible in the future for two reasons: (1) A further increase in data volumes, among other things due to the use of mobile low-cost sensors within the growing citizen science community (Koedel et al., 2022). (2) Sensors that move in space (i.e. on trains, airplanes etc.) will significantly increase the algorithmic complexity of the respective QC workflows. Therefore, we will assess semi-automated parameterization schemes as proposed by Durre et al. (2008) and Taylor and Loescher (2013) as solutions to growing volumes and complexity. In the long run, it is most likely that entirely automated and machine learning-based procedures will be required. These have recently been shown to achieve high classification accuracy by exploiting spatio-temporal patterns in the data of the entire sensor network (e.g. Erhan et al., 2021). A (partial) integration of machine-learning based QC-workflows as recently made available through the Python package *pyhydroqc* (Jones et al., 2022) will be evaluated. Furthermore, we will investigate methods to continuously monitor the correct functioning and performance of QC-workflows by analyzing spatial and temporal flagging patterns. This could be achieved based on methods as presented in Durre et al. (2008) and Smith et al. (2014). The monitoring results, i.e. measures of QC performance, could be communicated in a user-focused manner as presented in Sturtevant et al. (2021).

SaQC will be disseminated in the scientific community through hands-on workshops to enable more domain scientists to use it in their specific applications. With a growing user community, the authors envision an open source community that contributes specific QC-tests

```
# Multivariate spike detection using kNN and STRAY
# Normalization of variables for kNN
level_norm       ; transform(field="level", func="zScore", freq="30D")
water_temp_norm  ; transform(field="water_temp", func="zScore", freq="30D")
sac254_norm      ; transform(field="sac254_corr", func="zScore", freq="30D")

# Flagging by kNN and STRAY
kNN_scores ; assignKNNScore(field=["level_norm", "water_temp_norm", "sac254_norm"], freq="30D")
kNN_scores ; flagByStray(freq="30D", alpha=0.3)

# Project results of STRAY-algorithm onto variables
level       ; concatFlags(field="kNN_scores")
water_temp  ; concatFlags(field="kNN_scores")
sac254_corr ; concatFlags(field="kNN_scores")
```

Box III.

along with suitable parameterization schemes for future re-use. By providing a domain-agnostic and flexible software tool, the authors hope to contribute to an urgent and inevitable standardization of QC workflows and flagging schemes across domains, thus fostering FAIR data flows.

5. Conclusions

In the light of increasingly growing data volumes in the environmental sciences, rigorous automation of data workflows and their quality-control component is essential. This requires domain-agnostic software tools to establish automated QC-workflows, aiming at a user group that mostly consists of domain experts with limited IT-expertise. In this publication, we present SaQC, an Open Source Python package for the quality control of environmental time series data with the following features: It is universal regarding its application domain and flexible with regard to both its input/output data structures as well as flagging schemes. While providing a comprehensive set of methods, it can be extended in its pre- and post-processing as well as QC-testing functions. All these functions are designed in such a way so that the handling of quality flags between processing steps is controllable and explicit. In addition, it provides functionality to make QC-workflows traceable and reproducible, thus promoting FAIR workflows. SaQC is designed to be accessible, particularly to users without programming knowledge as is often the case in practical application. By means of a detailed software description and the presentation of two real-world applications, we showcase how SaQC can be used to process and perform quality control on environmental time series data both at a basic and at an advanced level. Consequently, we provide the scientific community with a software package that can automatically deliver environmental data that are of high quality, up-to-date, potentially large in volume and reusable, which are imperative these days for data-driven and sustainable science to address the urgent environmental challenges of our time.

CRediT authorship contribution statement

Lennart Schmidt: Conceptualization, Software development, Writing. **David Schäfer:** Conceptualization, Software development, Supervision. **Juliane Geller:** Conceptualization, Software development, Writing. **Peter Lünenschloss:** Conceptualization, Software development, Writing. **Bert Palm:** Conceptualization, Software development, Writing. **Karsten Rinke:** Data collection and curation, Writing, Review. **Corinna Rebmann:** Conceptualization, Data collection and curation, Review. **Michael Rode:** Conceptualization, Data collection and curation, Review. **Jan Bumberger:** Conceptualization, Writing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data & code are available on Zenodo https://doi.org/10.5281/zenodo.8092119.

Declaration of Generative AI and AI-assisted technologies in the writing process

In the final stage of the review process of this work, the authors used OpenAI ChatGPT-4 in order to improve legibility of individual phrases as well as the first paragraph of Section 2.1. After using this tool, the authors reviewed and edited the content as required and take full responsibility for the content of the publication.

Acknowledgments

We thank the Helmholtz Association and the Federal Ministry of Education and Research (BMBF) for supporting the DataHub Initiative of the Research Field Earth and Environment. The DataHub enables an overarching and comprehensive research data management, following FAIR principles, for all Topics in the Program Changing Earth – Sustaining our Future.

The Hohes Holz and the Rappbode observatories are financially supported by TERENO (Terrestrial Environmental Observatories, funded by the Helmholtz Association, the Federal Ministry of Education and Research (BMBF) and the Talsperrenbetrieb Sachsen-Anhalt.

Karsten Rinke received funding from the H2020-MSCA-ITN-2020 innovative training network InventWater (GrantNo 956623).

We acknowledge previous work carried out by Juliane Mai and Matthias Cuntz who established first data quality control procedures at the Hohes Holz site. We also thank Uwe Kiwel for support with the TERENO high-frequency monitoring and maintenance of the sensor measurements in the Lower Bode.

The authors wish to thank the anonymous reviewers for valuable comments that helped to improve the quality of this work.

```
varname     ; test
#-----------;-----------------------------------
# 1: Flagging of maintenance intervals
sac254      ; flagManual(mdata="maint")

# 2: Basic QC: Range-tests
level       ; flagRange(min=0)
water_temp  ; flagRange(min=-.5)
sac254      ; flagRange(min=0, max=70)

# 3: Pre-processing: Resampling via linear interpolation
level       ; align(freq="15min")
water_temp  ; align(freq="15min")
sac254      ; align(freq="15min")

# 4: Drift correction
sac254_corr  ; correctDrift(
                  field="sac254", maintenance_field="maint", model="exponential")

# 5: Multivariate spike detection using kNN and STRAY
# Normalization of variables for kNN
level_norm       ; transform(field="level", func="zScore", freq="30D")
water_temp_norm  ; transform(field="water_temp", func="zScore", freq="30D")
sac254_norm      ; transform(field="sac254_corr", func="zScore", freq="30D")

# Flagging by kNN and STRAY
kNN_scores  ; assignKNNScore(
                 field=["level_norm", "water_temp_norm", "sac254_norm"], freq="30D")
kNN_scores  ; flagByStray(freq="30D", alpha=0.3)

# Project results of STRAY-algorithm onto variables
level       ; concatFlags(field="kNN_scores")
water_temp  ; concatFlags(field="kNN_scores")
sac254_corr ; concatFlags(field="kNN_scores")
```

Box IV.

Appendix A

A.1. Config-file application 1

This config-file includes all steps to perform QC as presented in use application 1 in Section 3.1. For a more detailed description, the reader is referred to the respective Jupyter notebook https://doi.org/10.5281/zenodo.8092119

```
varname     ; test
#-----------;-----------------------------------
SM2         ; align(freq="15Min", method="nshift")
SM2         ; flagRange(min=10, max=60)
SM2         ; flagMAD(window="30D", z=3.5)
SM2         ; interpolate(limit=2)
```

A.2. Config-file application 2

Please note that, while the description of application 2 in Section 3.4 focuses on the variable *sac254* for legibility, this config-file lists the required steps for all three variables (*water level, water temperature* and *sac254*). For a more detailed description, the reader is referred to the respective Jupyter notebook (https://doi.org/10.5281/zenodo.8092119) (see Box IV).

References

Campbell, J.L., Rustad, L.E., Porter, J.H., Taylor, J.R., Dereszynski, E.W., Shanley, J.B., Gries, C., Henshaw, D.L., Martin, M.E., Sheldon, W.M., et al., 2013. Quantity is nothing without quality: Automated QA/QC for streaming environmental sensor data. BioScience 63 (7), 574–585. http://dx.doi.org/10.1525/bio.2013.63.7.10.

Crystal-Ornelas, R., Varadharajan, C., Christianson, D., Damerow, J., Weierbach, H., Robles, E., Ramakrishnan, L., Faybishenko, B., Pastorello, G., 2021. A library of AI-assisted FAIR water cycle and related disturbance datasets to enable model training, parameterization and validation. http://dx.doi.org/10.2172/1769646, URL https://www.osti.gov/biblio/1769646.

Doraiswamy, P., Pasteris, P., Jones, K., Motha, R., Nejedlik, P., 2000. Techniques for methods of collection, database management and distribution of agrometeorological data. Agricult. Forest Meteorol. 103 (1–2), 83–97. http://dx.doi.org/10.1016/S0168-1923(00)00120-9.

Durre, I., Menne, M.J., Vose, R.S., 2008. Strategies for evaluating quality assurance procedures. J. Appl. Meteorol. Climatol. 47 (6), 1785–1791. http://dx.doi.org/10.1175/2007JAMC1706.1.

Erhan, L., Ndubuaku, M., Di Mauro, M., Song, W., Chen, M., Fortino, G., Bagdasar, O., Liotta, A., 2021. Smart anomaly detection in sensor systems: a multi-perspective review. Inf. Fusion 67, 64–79. http://dx.doi.org/10.1016/j.inffus.2020.10.001.

Fiebrich, C.A., Morgan, C.R., McCombs, A.G., Hall Jr., P.K., McPherson, R.A., 2010. Quality assurance procedures for mesoscale meteorological data. J. Atmos. Ocean. Technol. 27 (10), 1565–1582. http://dx.doi.org/10.1175/2010JTECHA1433.1.

Foundation, A.S., 2014. Apache airflow. URL https://airflow.apache.org.

Gandin, L.S., 1988. Complex quality control of meteorological observations. Mon. Weather Rev. 116 (5), 1137–1156.

GoFair, 2021. FAIR principles. URL https://www.go-fair.org/fair-principles/.

Gong, A., Campbell, J., 2022. Great expectations. http://dx.doi.org/10.5281/zenodo.6403212.

Horsburgh, J.S., Reeder, S.L., Jones, A.S., Meline, J., 2015. Open source software for visualization and quality control of continuous hydrologic and water quality sensor data. Environ. Model. Softw. 70, 32–44. http://dx.doi.org/10.1016/j.envsoft.2015.04.002.

Huang, J., Borchardt, D., Rode, M., 2022. How do inorganic nitrogen processing pathways change quantitatively at daily, seasonal, and multiannual scales in a large agricultural stream? Hydrol. Earth Syst. Sci. 26 (22), 5817–5833.

Iglewicz, B., Hoaglin, D., 1993. Volume 16: how to detect and handle outliers. In: The ASQC Basic References in Quality Control: Statistical Techniques. Vol. 16, ASQC Quality Press Milwaukee (WI, USA).

Jones, A.S., Horsburgh, J.S., Eiriksson, D.P., 2018. Assessing subjectivity in environmental sensor data post processing via a controlled experiment. Ecol. Inform. 46, 86–96. http://dx.doi.org/10.1016/j.ecoinf.2018.05.001.

Jones, A.S., Jones, T.L., Horsburgh, J.S., 2022. Toward automating post processing of aquatic sensor data. Environ. Model. Softw. 151, 105364. http://dx.doi.org/10.1016/j.envsoft.2022.105364, URL https://www.sciencedirect.com/science/article/pii/S1364815222000706.

Kaffashzadeh, N., Kleinert, F., Schultz, M.G., 2019. A new tool for automated quality control of environmental time series (AutoQC4Env) in open web services. In: International Conference on Business Information Systems. Springer, pp. 513–518.

Koedel, U., Schuetze, C., Fischer, P., Bussmann, I., Sauer, P.K., Nixdorf, E., Kalbacher, T., Wichert, V., Rechid, D., Bouwer, L.M., Dietrich, P., 2022. Challenges in the evaluation of observational data trustworthiness from a data producers viewpoint (FAIR+). Front. Environ. Sci. 9, http://dx.doi.org/10.3389/fenvs.2021.772666.

Loescher, H.W., Kelly, E.F., Lea, R., 2017. National ecological observatory network: Beginnings, programmatic and scientific challenges, and ecological forecasting. In: Terrestrial Ecosystem Research Infrastructures. CRC Press, pp. 27–52.

Mollenhauer, H., Kasner, M., Haase, P., Peterseil, J., Wohner, C., Frenzel, M., Mirtl, M., Schima, R., Bumberger, J., Zacharias, S., 2018. Long-term environmental monitoring infrastructures in europe: observations, measurements, scales, and socio-ecological representativeness. Sci. Total Environ. 624, 968–978. http://dx.doi.org/10.1016/j.scitotenv.2017.12.095.

Petzold, A., Thouret, V., Gerbig, C., Zahn, A., Brenninkmeijer, C.A., Gallagher, M., Hermann, M., Pontaud, M., Ziereis, H., Boulanger, D., et al., 2015. Global-scale atmosphere monitoring by in-service aircraft–current achievements and future prospects of the European research infrastructure IAGOS. Tellus B: Chem. Phys. Meteorol. 67 (1), 28452. http://dx.doi.org/10.3402/tellusb.v67.28452.

Rebmann, C., Claudia, S., Sara, M.-J., Sebastian, G., Matthias, Z., Luis, S., Matthias, C., 2017. Integrative measurements focusing on carbon, energy and water fluxes at the forest site 'Hohes Holz' and the grassland 'Am Grossen Bruch'. In: EGU General Assembly Conference Abstracts. p. 9727.

Reznick, L., 1993. Using cron and crontab. Syst. Admin. 2 (4), 29–32.

Rinke, K., Kuehn, B., Bocaniov, S., Wendt-Potthoff, K., Büttner, O., Tittel, J., Schultze, M., Herzsprung, P., Rönicke, H., Rink, K., et al., 2013. Reservoirs as sentinels of catchments: the Rappbode reservoir observatory (Harz Mountains, Germany). Environ. Earth Sci. 69 (2), 523–536. http://dx.doi.org/10.1007/s12665-013-2464-2.

Rode, M., Wade, A.J., Cohen, M.J., Hensley, R.T., Bowes, M.J., Kirchner, J.W., Arhonditsis, G.B., Jordan, P., Kronvang, B., Halliday, S.J., Skeffington, R.A., Rozemeijer, J.C., Aubert, A.H., Rinke, K., Jomaa, S., 2016. Sensors in the stream: the high-frequency wave of the present. Environ. Sci. Technol. 50 (19), 10297–10307. http://dx.doi.org/10.1021/acs.est.6b02155.

Sheldon, W.M., 2008. Dynamic, rule-based quality control framework for real-time sensor data. In: Proceedings of the Environmental Information Management Conference. Citeseer, pp. 145–150.

Smith, D.E., Metzger, S., Taylor, J.R., 2014. A transparent and transferable framework for tracking quality information in large datasets. PLoS One 9 (11), e112249. http://dx.doi.org/10.1371/journal.pone.0112249.

Sturtevant, C., Metzger, S., Nehr, S., Foken, T., 2021. Quality assurance and control. In: Springer Handbook of Atmospheric Measurements. Springer, pp. 47–90. http://dx.doi.org/10.1007/978-3-030-52171-4_3.

Talagala, P.D., Hyndman, R.J., Leigh, C., Mengersen, K., Smith-Miles, K., 2019. A feature-based procedure for detecting technical outliers in water-quality fata from in situ sensors. Water Resour. Res. 55 (11), 8547–8568. http://dx.doi.org/10.1029/2019WR024906.

Talagala, P.D., Hyndman, R.J., Smith-Miles, K., 2021. Anomaly detection in high-dimensional data. J. Comput. Graph. Statist. 30 (2), 360–374. http://dx.doi.org/10.1080/10618600.2020.1807997.

Taylor, J., Loescher, H., 2012. Neon's fundamental instrument unit dataflow and quality assurance plan.

Taylor, J.R., Loescher, H.L., 2013. Automated quality control methods for sensor data: a novel observatory approach. Biogeosciences 10 (7), 4957–4971. http://dx.doi.org/10.5194/bg-10-4957-2013.

Vitale, D., Fratini, G., Bilancia, M., Nicolini, G., Sabbatini, S., Papale, D., 2020. A robust data cleaning procedure for eddy covariance flux measurements. Biogeosciences 17 (6), 1367–1391. http://dx.doi.org/10.5194/bg-17-1367-2020.

WMO, 2018. Guide to the Global Observing System (WMO-No.488). World Meteorological Organization Geneva, Switzerland.

Wollschläger, U., Attinger, S., Borchardt, D., Brauns, M., Cuntz, M., Dietrich, P., Fleckenstein, J.H., Friese, K., Friesen, J., Harpke, A., et al., 2017. The bode hydrological observatory: a platform for integrated, interdisciplinary hydro-ecological research within the TERENO harz/central german lowland observatory. Environ. Earth Sci. 76 (1), 1–25. http://dx.doi.org/10.1007/s12665-016-6327-5.

Zacharias, S., Bogena, H., Samaniego, L., Mauder, M., Fuß, R., Pütz, T., Frenzel, M., Schwank, M., Baessler, C., Butterbach-Bahl, K., et al., 2011. A network of terrestrial environmental observatories in Germany. Vadose Zone J. 10 (3), 955–973. http://dx.doi.org/10.2136/vzj2010.0139.